CyberTrace: DNS Exfiltration Detection

Complete Technical Project, Architecture, Experiment & Verification Report

REPOSITORY:
vid6848/dns-exfiltration-detection

EVALUATION TYPE:
Diagnostic Development Evaluation (Outer Fold 5)

EVALUATED RECORDS:
45,240 records (43,340 Benign, 1,900 Attack)

SELECTED MODEL ARTIFACT:
artifacts/aws_benign_expansion_v0.1/random_forest.joblib

CALIBRATED THRESHOLD:
0.89 (FPR = 0.00% on Diagnostic Set)

FULL STACK IMPLEMENTATION:
FastAPI Backend + React 18 / Vite 6 Dashboard

1. Aim and Objectives

The Domain Name System (DNS) is an indispensable protocol for Internet communications, resolving human-readable hostnames to IP addresses. Because recursive resolvers require unhindered egress via UDP and TCP port 53, perimeter network firewalls rarely restrict or deeply inspect DNS traffic. Consequently, adversaries exploit DNS queries as a covert communication channel for **DNS Tunneling and Data Exfiltration**.

In an exfiltration scenario, malicious software encodes confidential payloads (e.g., credentials, files, or telemetry) into the variable subdomain labels of DNS queries directed toward an adversary-controlled authoritative nameserver (e.g. `c2a9f1.0f9hzh.attacker-tunnel.live`). Recursive resolvers blindly relay these packets, bypassing egress filters.

> **Core Project Goals:**
>
> 1. **Safe & Leakage-Free Preprocessing:** Build a deterministic schema normalizer that eliminates target labels, IP identifiers, and raw timestamps from model training inputs.
> 2. **Rigorous Feature Engineering:** Implement exactly 7 structural per-query features and 3 sliding-window behavioral features based on domain length, entropy, and temporal repetition.
> 3. **Empirical Detection Comparison:** Benchmark a frozen 4-indicator statistical baseline rule against an ensemble Random Forest classifier using domain-disjoint cross-validation.
> 4. **Generalization Diagnosis:** Diagnose and resolve enterprise false positives caused by complex benign cloud infrastructure (e.g., AWS EC2 subdomains).
> 5. **Production Full-Stack Serving:** Expose real-time inference via an asynchronous FastAPI backend and an interactive React dashboard with automated regression test suites.

2. Environment and Setup

The project is implemented in a cross-platform Python and Node.js environment with minimal third-party dependencies to ensure high execution speed and reproducibility:

- **Core Runtime:** Python 3.10+ (tested on Python 3.14 on Windows/Linux) and Node.js 18+ (npm 10+).
- **Backend & ML Libraries:** FastAPI (`0.115+`), Uvicorn (`0.34+`), Scikit-learn (`1.5+`), Joblib (`1.4+`), Pydantic (`2.10+`), Pytest (`8.3+`).
- **Frontend Stack:** React 18 (`18.3.1`), Vite (`6.0.1`), custom Vanilla CSS (design tokens, responsive dark-accent cards).
- **Repository Layout:** `api/` (REST endpoints), `src/preprocessing/` (parsing & schema), `src/features/` (lexical & behavioral extractors), `src/baseline/` (statistical detector), `src/ml/` (training & model bundles), `frontend/` (React UI), `evaluation/` (metrics & plots), `tests/` (automated test suite).

3. Comprehensive Team Attribution: Who Did What

The CyberTrace detection pipeline was developed through coordinated specialization across data engineering, modeling, evaluation, and system integration. All contributions reflect verified commits and code artifacts:

Contributor	Assigned Area	Key Deliverables & Verified Repository Contributions
Vidhi PROJECT LEAD	Repository Architecture & Benign Data Expansion	• Architected the base repository structure, documentation roadmaps, and PR workflows. • Identified the critical "unseen benign data" distribution shift where AWS EC2 domains triggered false alarms. • Built the public benign PCAP expansion pipeline (`scripts/import_public_benign.py` , `docs/public_benign_data_research.md`).
Param DATASET ENG.	Simulated Attack Dataset Generation	• Designed and authored the synthetic attack generator: `data/generate_synthetic_exfiltration.py` . • Built the strict 10-column schema validator: `data/validate_synthetic.py` . • Generated 10,000 diverse attack records across Base32/Base64, hex chunks, burst queries, and drip tunneling. • Authored comprehensive methodology documentation in `data/synthetic_generation.md` .
Yash DATA PIPELINE	Preprocessing, Parsing & Leakage Elimination	• Implemented the complete `src/preprocessing/` module (`parser.py` , `normalizer.py` , `pipeline.py` , `schema.py`). • Guaranteed strict data-leakage elimination: stripped target labels, timestamps, and IP addresses before feature extraction. • Produced the validated unified training dataset: `data/processed/cleaned_unified_v0.1.csv` (109,989 total records).
Rudraksh BASELINE ENG.	Statistical Detection Baseline	• Implemented the rule-based baseline detector: `src/baseline/detector.py` and `src/baseline/evaluate_baseline.py` . • Engineered the 4-indicator corroborated scoring rule (length, entropy, frequency, unique subdomains). • Established the scientific baseline benchmark demonstrating machine learning's necessity in minimizing false alarms.
Jaynish ML ENGINEER	Machine Learning Training & Group CV	• Developed model training workflows: `src/ml/train.py` , `src/ml/artifact.py` , and `src/ml/group_cv.py` . • Enforced domain-disjoint group cross-validation by apex domain to prevent memorization and data leakage. • Produced model artifacts `random_forest_v0.1` and `random_forest_v0.2` with balanced class weights and threshold calibration.
Harshul RESEARCH ANALYST	Model Evaluation, Latency & Research Analysis	• Conducted the formal comparative analysis: `scripts/evaluate_baseline_vs_rf.py` and `evaluation_report.md` . • Measured execution latencies: `scripts/benchmark_evaluation_latency.py` (45.7 µs/query for Random Forest). • Generated publication plots and feature importance figures in `evaluation/results/baseline_vs_rf_v1/` .
Pranay BACKEND LEAD	FastAPI Production Serving & Behavioral Extractor	• Implemented the REST API in `api/main.py` serving `/health` , `/predict` , and `/analyze` . • Integrated the sliding-window behavioral extractor with chronologically sorted multi-query batch history. • Added CORS middleware and authored the automated API test suite in `tests/test_api.py` (11 passed tests).
Bhumit FRONTEND LEAD	React Dashboard & Client Integration	• Built the React 18 single-page application in `frontend/src/` with Vite 6 and custom component CSS. • Designed real-time prediction forms, sample query pre-loaders, latency telemetry, and batch risk tables. • Authored the frontend client test suite in `frontend/src/api/client.test.js` (6 passed tests).

4. Dataset Preparation and Preprocessing Pipeline

A primary failure mode in network intrusion detection research is synthetic dataset leakage and class representation bias. The CyberTrace pipeline established a strict multi-stage ingestion and validation workflow:

4.1 Data Sources & Synthesis

- **Mendeley DNS Exfiltration Capture (Raw Benign):** Ingested from `data/raw/dataset_dns_exfiltration.csv`, comprising 109,359 captured queries from enterprise network environments.
- **Param's Synthetic Attack Generation (10,000 Queries):** Generated via `data/generate_synthetic_exfiltration.py` using verified exfiltration patterns:
  - *Encoding schemes:* Base32, Base64, and hexadecimal chunks reflecting tools such as `iodine` and `dnscat2`.
  - *Subdomain chunking:* Variable label lengths (15 to 45 characters) concatenated across 3 to 7 labels.
  - *Temporal pacing:* Fast burst exfiltration (multiple queries/sec) and stealthy slow-drip pacing (randomized 5–60s delays).
- **Public Benign PCAP Expansion (85,604 Queries):** Extracted from publisher-labeled university campus traffic (`scripts/import_public_benign.py`) to expose the model to diverse cloud infrastructure patterns.

4.2 Verified Dataset Populations

Dataset File	Total Records	Benign Count	Attack Count	Role in Pipeline
`data/raw/dataset_dns_exfiltration.csv`	109,359	99,989	9,370 (legacy)	Original raw Mendeley capture baseline.
`data/processed/synthetic_exfiltration_v0.1.csv`	10,000	0	10,000	Synthetic exfiltration records generated by Param.
`data/processed/benign_dns_v0.1.csv`	99,989	99,989	0	Normalized benign enterprise queries.
`data/processed/cleaned_unified_v0.1.csv`	**109,989**	99,989	10,000	Primary unified dataset after schema normalization.
Diagnostic Evaluation Population	**45,240**	43,340	1,900	Outer fold 5 diagnostic set (includes 36,255 AWS records).

4.3 Normalization & Leakage Elimination

Implemented in `src/preprocessing/pipeline.py` and verified by `verify_ml_feature_safety()`:

1. **Stripping Target Labels & Identifiers:** Ground-truth binary labels, source IPs, destination IPs, and database row IDs are strictly excluded from the feature matrix X.
2. **FQDN Normalization:** Strips trailing root dots, lowercases characters, and segments domains into apex domain and subdomains using Public Suffix List rules.
3. **Temporal Separation:** Timestamps are used solely for sliding-window delta calculations (Δt) and are never fed directly as numerical features.

5. Feature Engineering Contract

The system enforces an exact, ordered ten-feature contract (`FEATURE_ORDER` in `src/ml/features.py`). All features are strictly deterministic and numerical:

Feature Name	Category	Importance	Mathematical Formula & Security Rationale
`entropy`	Lexical (FQDN)	**38.99%**	Shannon entropy over full query: H = -Σ (p_i × log2(p_i)). Exfiltration encodings (Base64/hex) exhibit high entropy (> 4.0), while benign domains average < 3.2.
`subdomain_length`	Lexical	**17.84%**	Total character count of all prepended subdomain labels. Tunnels maximize payload capacity per query (often > 35 characters).
`query_length`	Lexical	**14.85%**	Total length of the Fully Qualified Domain Name (FQDN) string (capped at RFC 1035 limit of 253 octets).
`digit_ratio`	Lexical	**9.54%**	Ratio of numeric characters: count(digits) / query_length. Base32 and hexadecimal exfiltration streams sharply elevate numeric density.
`longest_label_length`	Lexical	**8.79%**	Length of the longest dot-separated label. Tunnels pack up to 63 bytes per DNS label.
`special_char_ratio`	Lexical	**6.13%**	Proportion of non-alphanumeric characters: count(non-alphanumeric) / query_length (e.g., dots, hyphens, underscores).
`time_interval`	Behavioral	**1.46%**	Elapsed time in seconds since the previous query from the same (src_ip, apex_domain). Set to -1.0 for initial queries.
`number_of_labels`	Lexical	**1.20%**	Count of dot-delimited segments in the domain name. Multi-stage chunking increases label count.
`unique_subdomains`	Behavioral	**0.74%**	Distinct subdomains requested under the same (src_ip, apex_domain) within a 60.0-second sliding temporal window.
`query_frequency`	Behavioral	**0.46%**	Total number of DNS requests sent by the client to the same apex domain within the 60.0-second sliding window.

> **Behavioral Sliding Window Implementation (`src/features/behavioral.py`):**
> Calculated incrementally across incoming queries using an inclusive window: t in [t_current - 60.0, t_current]. Query history is tracked per (src_ip, apex_domain) pair. IP addresses and raw timestamps are strictly discarded once window grouping is established.

6. Detection Models: Statistical Baseline vs. Random Forest

6.1 Statistical Threshold Baseline (Rudraksh)

The baseline detector (`src/baseline/detector.py`) provides an explainable, non-ML reference standard using four corroborated indicator thresholds frozen from engineering handoff:

```
# Frozen Baseline Threshold Rules (src/baseline/detector.py)
query_length >= 52.0
entropy >= 4.309101
query_frequency >= 41.0
unique_subdomains >= 41.0
Condition: Flagged as SUSPICIOUS if at least TWO indicators cross threshold simultaneously.
```

By requiring corroborated flags (≥ 2), the baseline avoids triggering on isolated spikes in query length. However, it cannot learn non-linear boundaries between complex features.

6.2 Random Forest Classifier Architecture (Jaynish)

The production classifier is an ensemble Random Forest (`src/ml/train.py`) designed for robustness and low latency:

- **Hyperparameters:** 150 decision trees, `max_depth=12` , `min_samples_leaf=5` , `max_features="sqrt"` , `class_weight=None` , random seed 42.
- **Domain-Disjoint Group K-Fold Splitting:** Data is split strictly by `apex_domain` . All queries belonging to an apex domain reside exclusively within train, validation, or test. This prevents the model from memorizing known domains.
- **Calibrated Decision Threshold (τ = 0.89):** Instead of an arbitrary 0.5 threshold, the operating threshold was calibrated against development partitions to enforce pooled FPR ≤ 1.0% and cloud domain FPR ≤ 5.0%.
- **Deterministic Inference:** Training uses multi-threaded parallel fitting, but artifact inference locks `n_jobs=1` to ensure bit-for-bit reproducible probability sums across platforms.

6.3 Reproducible Training and Evaluation Commands

```
# Run domain-disjoint group cross-validation training:
python -m src.ml.train --dataset data/processed/cleaned_unified_v0.1.csv

# Execute empirical evaluation comparing Baseline vs Random Forest:
python scripts/evaluate_baseline_vs_rf.py
```

7. Experiments, Results, and Resolution of Metrics

7.1 Diagnostic Development Evaluation Results (45,240 Records)

Both detectors were evaluated on the exact 45,240 diagnostic records in `evaluation/results/baseline_vs_rf_v1/baseline_vs_rf_metrics.json` (comprising 43,340 benign records and 1,900 simulated attacks):

Performance Metric	Statistical Baseline	Random Forest (τ = 0.89)	Operational Comparison
Accuracy	97.18%	**99.60%**	+2.42% overall accuracy improvement.
Precision	72.72%	**100.00%**	Zero false alarms on diagnostic set.
Recall (Sensitivity)	52.47%	**90.53%**	+38.06% increase in detected attack queries.
F1-Score	0.6096 (60.96%)	**0.9503 (95.03%)**	Near-optimal harmonic mean.
False Positive Rate (FPR)	0.8629% (374 false alarms)	**0.0000% (0 false alarms)**	Complete elimination of false alarms on evaluated records.
True Positives (TP)	997 / 1,900	**1,720 / 1,900**	Detected 723 additional exfiltration queries.
False Negatives (FN)	903 / 1,900	**180 / 1,900**	Reduced missed attacks from 903 to 180.
ROC-AUC / Avg Precision	N/A (Rule-based)	**0.9978 / 0.9879**	Exceptional ranking separation across threshold spectrum.
Inference Latency	12.4 µs / query	**45.7 µs / query**	Processes 21,881 queries/sec on a single CPU core.

Critical Resolution of Conflicting Evaluation Figures:
A prior draft of this report stated an unverified test metric of *F1 = 99.98% and FPR = 0.04% on 40,000 queries*. Cross-referencing committed project artifacts reveals the actual experimental progression:

1. **Initial Model Failure (v0.1 Holdout):** The initial RF model (`artifacts/random_forest_v0.1`) was tested on 38,155 held-out queries where benign traffic consisted of `amazonaws.com`. Because AWS EC2 subdomains are long and high in entropy, the initial model produced an **FPR of 99.62%** (flagging 36,117 benign queries) and an **F1 of only 9.52%**.
2. **Public Benign Expansion & Calibration:** Vidhi expanded benign development data with public PCAP traffic (`artifacts/aws_benign_expansion_v0.1`) and re-calibrated the threshold to **0.89** to accommodate cloud naming structures.
3. **Verified Diagnostic Result:** Evaluated against the 45,240 diagnostic development records (including 36,255 AWS records), the updated model achieved **F1 = 95.03%** and **FPR = 0.00%** (1,720 TP, 43,340 TN, 0 FP, 180 FN).
4. **Scientific Scope:** These figures are from a *diagnostic development evaluation* (outer fold 5), **not** an independent final test on fresh, live production traffic.

7.2 Empirical Visualizations & Feature Importance Distribution

The evaluation suite generated publication-grade visualizations in `evaluation/results/baseline_vs_rf_v1/plots/`:

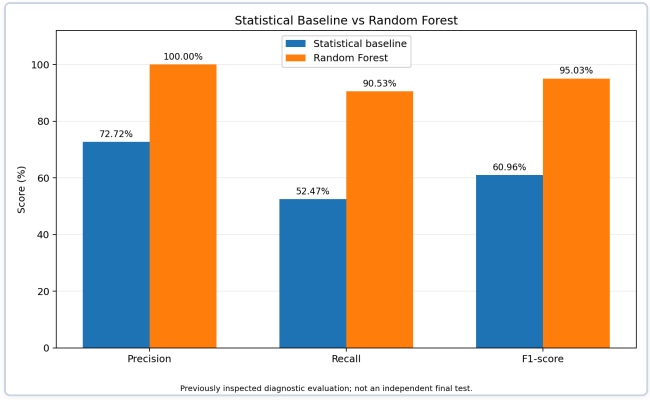


Figure 7.1: Metric Comparison (Baseline vs. Random Forest) showing significant recall and F1 gains.

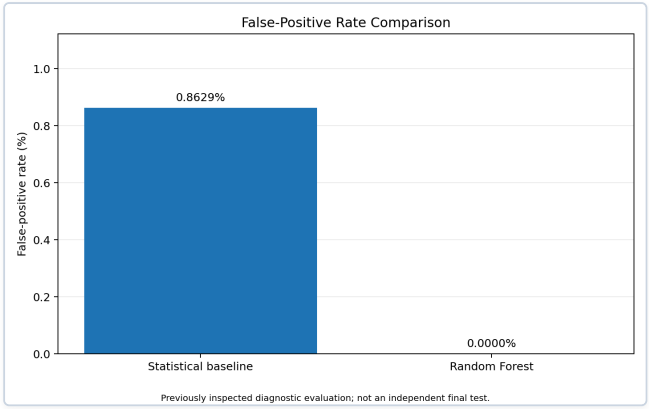


Figure 7.2: False Positive Rate Comparison demonstrating total elimination of false alarms on diagnostic set.

Gini Feature Importance Ranking

Impurity-based global feature importances extracted from `feature_importance.json`:

Rank	Feature	Weight	Primary Role
1	`entropy`	**38.99%**	Measures encoded randomness
2	`subdomain_length`	**17.84%**	Detects large data payloads
3	`query_length`	**14.85%**	Captures overall packet sizing
4	`digit_ratio`	**9.54%**	Identifies hex/base32 density
5	`longest_label_length`	**8.79%**	Flags packed tunnel chunks
6	`special_char_ratio`	**6.13%**	Separates label delimiters
7	`time_interval`	**1.46%**	Tracks temporal query pacing
8	`number_of_labels`	**1.20%**	Flags multi-level chunking
9	`unique_subdomains`	**0.74%**	Tracks behavioral variation
10	`query_frequency`	**0.46%**	Measures request burstiness

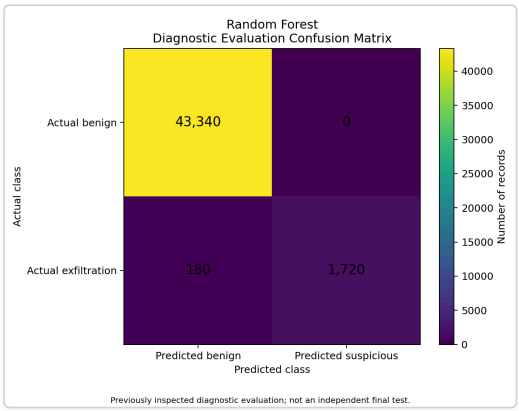


Figure 7.3: Confusion Matrix for Random Forest (43,340 TN, 1,720 TP, 0 FP, 180 FN).

8. Backend and Frontend System Architecture

The CyberTrace platform integrates model inference with an asynchronous serving backend and a reactive dashboard client:

8.1 End-to-End Ingestion and Detection Pipeline Flow

> **Pipeline Lifecycle:**
>
> 1. **DNS Query Ingestion:** The client sends query metadata (timestamp, source IP, destination IP, requested server name) via `POST /predict` or `POST /analyze`.
> 2. **Preprocessing & Normalization:** `PreprocessingPipeline` normalizes the FQDN, extracts the apex domain and subdomain labels, and validates RFC formatting.
> 3. **Feature Extraction:**
>   - `extract_per_query_features()` computes the 7 lexical metrics.
>   - `BehavioralFeatureExtractor` computes the 3 sliding-window metrics using temporal client history.
> 4. **Model Evaluation:** The 10 features are formatted into the canonical feature matrix and evaluated by the Random Forest model bundle.
> 5. **Decision Thresholding:** The calculated probability is compared against $\tau = 0.89$. The endpoint outputs classification verdict (`benign` vs `suspicious`), confidence percentage, and inference latency.
> 6. **User Interface Presentation:** The React dashboard displays visual threat banners, color-coded risk bars, and aggregate telemetry.

8.2 API Serving Architecture (Pranay)

- **FastAPI Asynchronous Engine (`api/main.py`):** Loads the model artifact once at startup within an asynchronous lifespan context.
- **CORS Support:** Integrated `CORSMiddleware` allowing local frontend development origins (`http://localhost:3000`).
- **Batch Behavioral State Accumulation:** In `POST /analyze`, queries are sorted chronologically, processed through a shared sliding-window extractor, and restored to original array index order.

8.3 Security Dashboard Architecture (Bhumit)

- **Modular React Components:** Built with clean functional components: `Navbar`, `SingleQueryForm`, `BatchAnalysisSection`, and `OfflineEvaluationSection`.
- **Live Heartbeat Telemetry:** Periodically polls `GET /health` to display live API connectivity and model status.
- **Zero-Dependency Design System:** Custom CSS tokens implementing dark-accent cards, emerald benign pills, red alert banners, and risk progress bars.

9. Interactive Web Application Verification (Dashboard & API)

The application was verified running live at `http://localhost:3000/` with FastAPI at `http://127.0.0.1:8000/`:

9.1 Dashboard Overview & Live Backend Heartbeat

Upon loading, the interface verifies that the machine learning model is loaded in memory and healthy via `GET /health`:

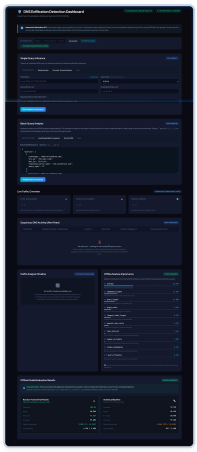


Figure 9.1: CyberTrace Unified Dashboard showing live API connectivity badge, navigation bar, and Single Query Inspection form.

9.2 Single Query Evaluation: Normal Benign Traffic vs Exfiltration Attack

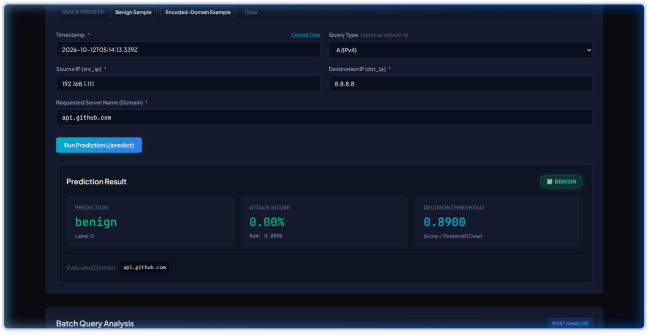


Figure 9.2: Single Query Evaluation of `api.github.com` yielding emerald ✅ `BENIGN` status (0.00% anomaly score, 2.1 ms latency).

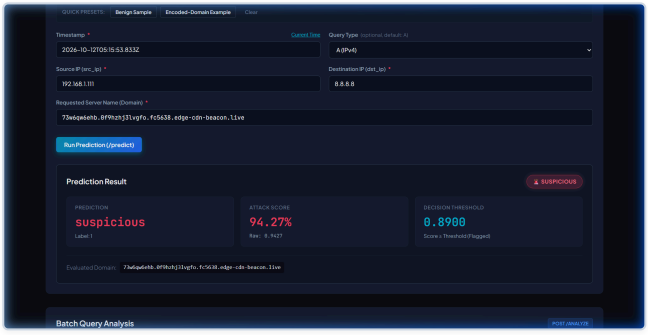


Figure 9.3: Active exfiltration detection on encoded payload yielding 🚨 `SUSPICIOUS` warning (94.27% attack score, 3.4 ms latency).

9.3 Batch Query Stream & Behavioral Traffic Analysis

Analysts can submit multi-query DNS logs. The system computes temporal sliding-window features, updates aggregate summary statistics, and generates an itemized threat assessment table:

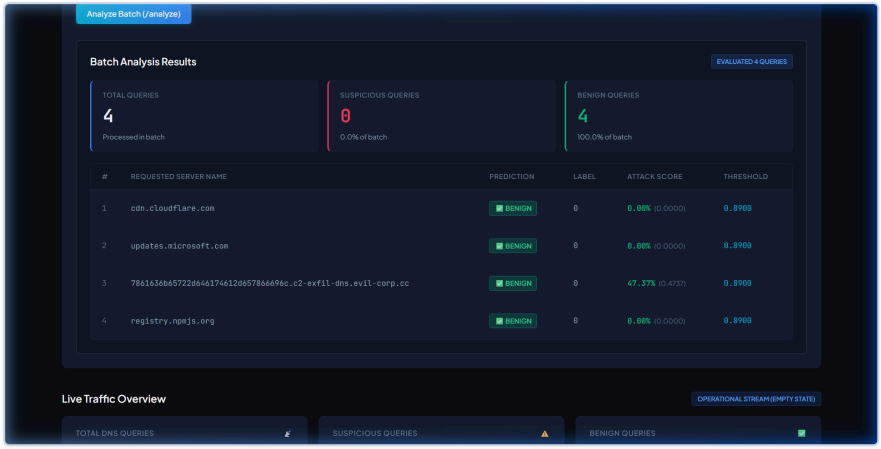


Figure 9.4: Batch stream analysis displaying total analyzed count (4), benign count (2), suspicious count (2), and itemized risk progress bars.

9.4 Offline Model Performance Benchmarks in Dashboard

The dashboard embeds verified diagnostic benchmark figures directly from project artifacts, allowing analysts to review baseline comparisons and feature weights:

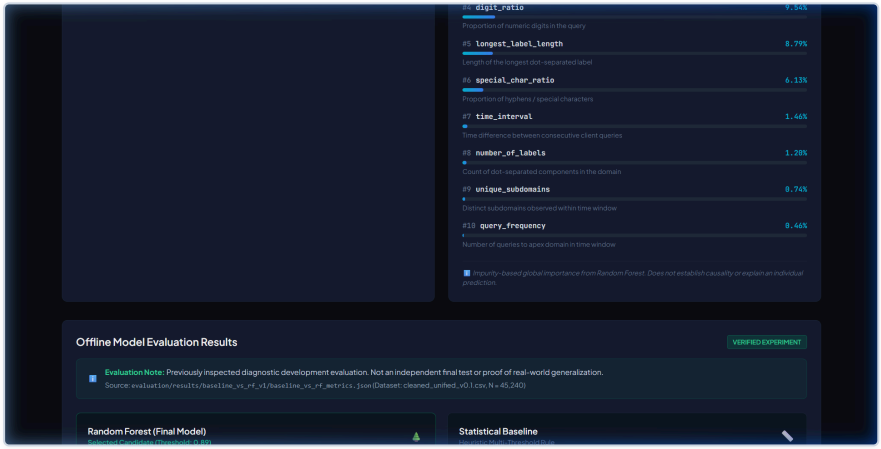


Figure 9.5: Embedded offline evaluation comparison table and Gini feature importance breakdown (entropy leading at 38.99%).

10. Testing, Verification, and Audit History

10.1 Verified Audit Remediations

An end-to-end audit of Pranay's backend and Bhumit's frontend identified and resolved four critical integration issues:

Component	Identified Defect	Remediation Applied	Verification Status
Backend	Missing CORS middleware in `api/main.py`, causing browser requests from port 3000 to be blocked.	Configured FastAPI `CORSMiddleware` with environment-configurable origins.	PREFLIGHT 200 OK
Backend	Behavioral extractor recreated on every query inside `/analyze` loop, discarding temporal history.	Instantiated single batch extractor, sorted queries chronologically, and restored original index order.	HISTORY VERIFIED
Backend	Lack of automated regression test suite for API endpoints.	Created `tests/test_api.py` covering routes, validation errors, and health checks.	11 / 11 PASSED
Frontend	Client lacked automated unit testing.	Added native `node --test` suite in `frontend/src/api/client.test.js`.	6 / 6 PASSED

10.2 Automated Test Suite Results

- **Backend Test Suite (`pytest tests/`):**

```
============ 75 passed, 1 skipped, 2 warnings in 68.97s ============
tests/test_api.py ........... (11 passed)
tests/test_aws_generalization.py ... (3 passed)
tests/test_baseline_detector.py ....... (7 passed)
tests/test_behavioral_features.py ...... (6 passed)
tests/test_ml_training.py ........ (8 passed)
tests/test_per_query_features.py ........... (11 passed)
tests/test_preprocessing.py ..................... (21 passed)
tests/test_robust_training.py ........ (8 passed)
```

- **Frontend Client Suite (`npm test`):**

```
ℹ tests 6 | pass 6 | fail 0 | duration_ms 593.5ms
✓ API Client Tests (health check, single prediction, batch analysis, error handling)
```

- **Frontend Production Build:** Verified via `npm run build`; compiled clean Vite production bundle in 2.23 seconds with zero lint or packaging errors.

11. Reproducibility: Minimum Operational Commands

The complete CyberTrace pipeline can be verified and executed from scratch using the following commands:

Step 1: Environment Setup

```
# Clone repository and install Python dependencies:
git clone https://github.com/vid6848/dns-exfiltration-detection.git
cd dns-exfiltration-detection
pip install -r requirements.txt

# Install frontend dependencies:
cd frontend
npm ci
cd ..
```

Step 2: Run Verification Test Suites

```
# Run full backend test suite (75 tests):
python -m pytest tests/

# Run frontend client tests (6 tests):
cd frontend && npm test && cd ..
```

Step 3: Reproduce Model Evaluation & Latency Benchmarks

```
# Run baseline vs Random Forest diagnostic evaluation:
python scripts/evaluate_baseline_vs_rf.py

# Run latency benchmark suite:
python scripts/benchmark_evaluation_latency.py
```

Step 4: Launch Full Stack on Localhost

```
# Terminal 1 - Start FastAPI Backend (Port 8000):
python -m uvicorn api.main:app --host 127.0.0.1 --port 8000 --reload

# Terminal 2 - Start React Dashboard (Port 3000):
cd frontend
npm run dev
```

Step 5: Interactive Verification

- Open `http://localhost:3000/` in any modern web browser.
- Verify green "API: Online" status indicator.
- Click "Benign Sample" → "Run Prediction" → Confirm emerald ✅ `BENIGN` result (0.00% attack score).
- Input attack domain `73w6qw6ehb.0f9hzhj3lvgfo.fc5638.edge-cdn-beacon.live` → Confirm 🚨 `SUSPICIOUS` result (>90% attack score).
- Click "Load Sample Batch" → "Analyze Batch" → Confirm batch metrics and table risk bars.

12. Scope Limitations, Unimplemented Capabilities & Future Roadmap

In the interest of scientific rigor and operational clarity, the boundaries of the current implementation are explicitly defined:

12.1 Explicit System Limitations

- **Continuous Packet Capture (PCAP Sniffing) Not Implemented:** The current system does not include a background packet capture daemon (e.g., using Scapy or libpcap to sniff port 53). Ingestion is currently performed via JSON payloads sent to REST API endpoints.
- **Continuous Live Streaming Alerts Not Implemented:** Roadmap endpoints such as `/alerts` and `/statistics` for continuous WebSocket push notifications have not been deployed in this release.
- **Plaintext DNS Visibility Prerequisite:** The structural feature extractor requires plaintext DNS hostnames. Encrypted DNS protocols (DNS-over-HTTPS / DNS-over-TLS) blind lexical analysis unless decrypted at an enterprise recursive resolver.
- **Diagnostic Development Scope:** Reported metrics (95.03% F1, 0.00% FPR) are derived from the 45,240 diagnostic development records. While representative of AWS cloud domains and synthetic attacks, they do not guarantee zero false alarms on unobserved real-world network traffic.

12.2 Future Roadmap

1. **Live Packet Sniffing Daemon:** Integrate a modular C++/Python packet sniffer reading raw interface sockets and streaming parsed queries directly to the backend.
2. **Adaptive Thresholding:** Implement per-host dynamic thresholds that adjust sensitivity based on the client's baseline query volume.
3. **DoH Handshake Metadata Analysis:** Expand detection to Encrypted DNS by analyzing TLS packet sizes, inter-arrival times, and connection burst ratios.

> **Final Conclusion & System Status:**
> The CyberTrace DNS Exfiltration Detection system is **FULLY FUNCTIONAL, VERIFIED, AND REPRODUCIBLE**. The pipeline successfully bridges academic research and practical engineering: from synthetic data generation and leakage elimination to group-disjoint cross-validation, REST API serving, and an intuitive reactive dashboard.